

Supplementary Materials: Random is Faster Than Systematic in Multi-Objective Local Search

Zimin Liang, Miqing Li*

School of Computer Science, University of Birmingham, Birmingham B15 2TT, U.K.
zxl525@student.bham.ac.uk, m.li.8@bham.ac.uk

Introduction

This supplementary document provides additional material for the main submission titled “*Random is Faster than Systematic in Multi-Objective Local Search*” submitted to the 40th annual AAAI conference on artificial intelligence.

The rest of the document is structured as follows. The first section introduces the multi-objective combinatorial optimisation problems considered in this paper. The second and third sections give the experimental settings and additional experimental results, respectively. The fourth section explains the procedure of investigating the distribution of good neighbours among solutions in the archive. Lastly, the fifth and sixth sections give the additional proofs regarding the two toy examples and the Best-from-Multiple-Selections mentioned in the main paper, respectively.

Multi-Objective Combinatorial Problems

We consider four MOCOPs, the multi-objective 0/1 knapsack [62], travelling salesman problem (TSP) [55], quadratic assignment problem (QAP) [38] and NK-landscapes [3]. Each problem was instantiated in three sizes (100, 200 and 500 variables).

Multi-Objective 0-1 Knapsack Problem (Knapsack). The multi-objective 0-1 knapsack problem [62] is a widely studied MOCOP. Given a set of D items $\mathbf{x} = (x_1, x_2, \dots, x_D) \in \{0, 1\}^D$, the m -objective problem is defined as the following.

$$\max f_j(x) = \sum_{i=1}^D v_{ji}x_i, \quad j = 1, \dots, m \quad \text{s.t.} \quad \sum_{i=1}^D w_i x_i \leq c \quad (1)$$

Here, $v_{ji} \geq 0$ is the value of the item i in objective j , w_i is the item’s weight, and $c = \frac{1}{2} \sum w_i$ is the capacity. Following [43], both v_{ji} and w_i are sampled uniformly from $\{10, 11, \dots, 100\}$.

Multi-Objective Travelling Salesman Problem (TSP). The multi-objective TSP extends the classical TSP, with multiple costs between each pair of cities [55], and aims to find the route minimising multiple travelling costs for

visiting all the cities exactly once, returning to the start. Formally, given a network $L = (V, C)$, where $V = \{v_1, \dots, v_D\}$ is a set of D nodes and $C = \{C_j : j \in \{1, \dots, m\}\}$ is a set of m cost matrices between nodes ($C_j : V \times V$), the problem is to find the Pareto optimal set of Hamiltonian cycles that minimise each of the m cost objectives.

Multi-Objective Quadratic Assignment Problem (QAP).

The multi-objective QAP [38] models facility-location assignments with multiple flow types. Given m cost matrices $[C_{1,i,j}], \dots, [C_{m,i,j}]$ and a distance matrix $[L_{u,v}]$, a solution is a permutation $x = (x_1, \dots, x_D)$ where x_i denotes the location of facility i . The problem is defined as the following.

$$\min f_k(x) = \sum_{i=1}^D \sum_{j=1}^D C_{k,i,j} L_{x_i, x_j}, \quad k = 1, \dots, m \quad (2)$$

Multi-Objective NK-Landscape. The multi-objective NK-landscapes [3] are widely used due to their tunable ruggedness [63]. Here, N represents the length of the bit-string and K represents the epistasis degree (i.e., each variable is influenced by K other variables, collectively referred to as its locus). For consistency, we denote the length of the bit-string as D . Then, in a m -objective NK-landscape problem, each objective f_j is defined as:

$$\max f_j(x) = \frac{1}{D} \sum_{i=1}^D c_{ij}(x_i, x_{k_{ij1}}, \dots, x_{k_{ijK}}), \quad j = 1, \dots, m. \quad (3)$$

Where c_{ij} represent the fitness contribution of the i -th variable, influenced by K other variables in its locus that collectively decide its contribution to the j -th objective. Each c_{ij} depends on the values of the i -variable and the variables in its locus, resulting in 2^{K+1} possible combinations of input and the corresponding output values. Each output is randomly sampled from $(0, 1]$. Following [1, 15], the K other variables of a variable’s locus are drawn independently and uniformly at random for each i (variable) and j (objective), resembling a random epistasis pattern.

Experimental Settings

In our experiments, for each MOCOP we consider three problem sizes (100, 200, and 500). It is important to note

*Corresponding author.

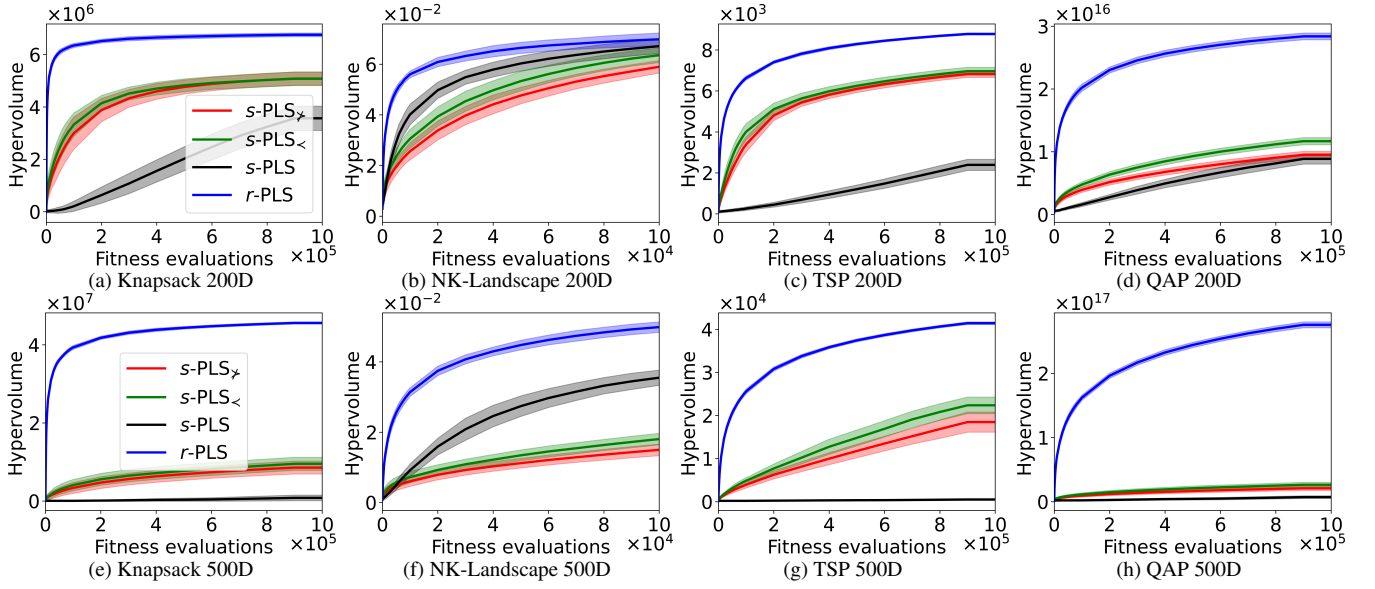


Figure 1: The hypervolume trajectory (higher is better) of the considered s -PLS, s -PLS $_x$, s -PLS $_<$ and r -PLS across 30 runs on the four MOCOPs with 200 variables (the top panel) and with 500 variables (the bottom panel). The bolded line and shaded area represent the mean and standard deviation of the hypervolume, respectively.

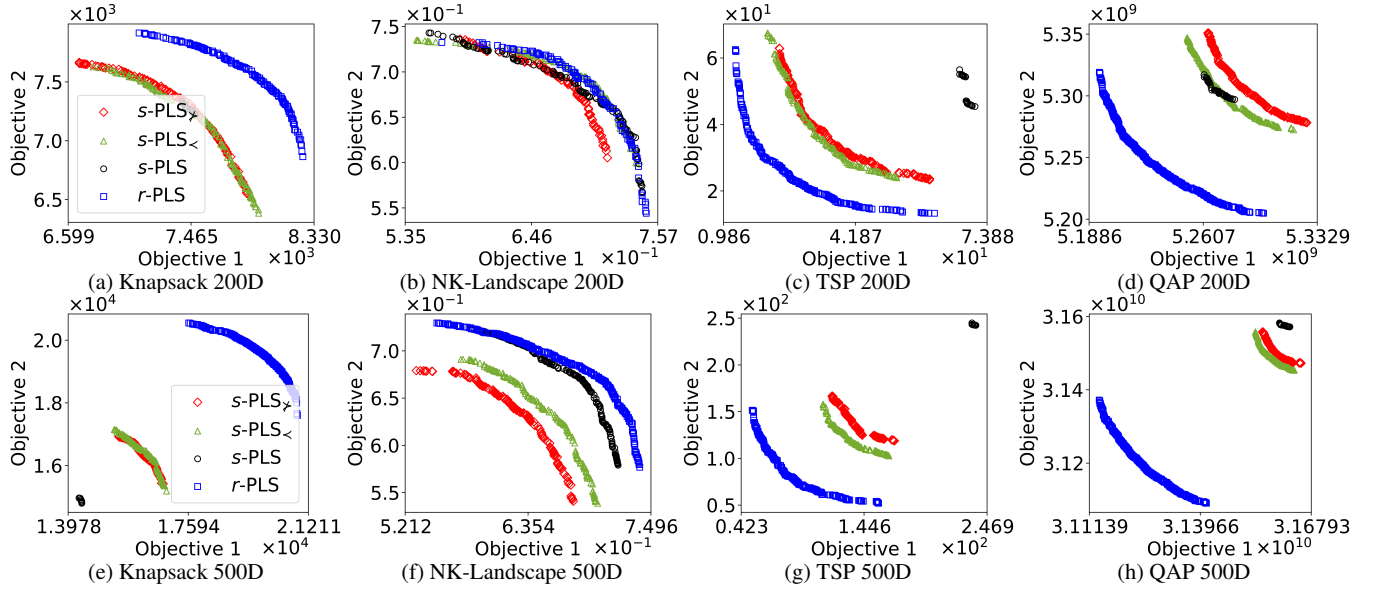


Figure 2: All non-dominated solutions (i.e., solutions in the archive) obtained by the considered s -PLS, s -PLS $_x$, s -PLS $_<$ and r -PLS in a typical run on the four MOCOPs with 200 variables (the top panel) and with 500 variables (the bottom panel), where the Knapsack and NK-Landscape are maximisation problems, and the TSP and QAP are minimisation problems.

that r -PLS and s -PLS typically use different stopping conditions: r -PLS runs until the evaluation budget is exhausted, whereas s -PLS may terminate earlier when there are no more unexplored solutions in the archive. To ensure fairness, all algorithms start from the same initial solution. For some situation, s -PLS can terminate rather early, for example, within 10^5 evaluations. In such cases, we stop r -PLS when s -PLS terminates. All algorithms are implemented in Java using the jMetal platform [22]. For each problem instance, each algorithm is executed over 30 independent runs, with each run

performed on a separate core of an AMD Ryzen R9-7945HX CPU, with 16 GB of memory allocated per run.

Neighbourhood operators are matched to the encoding of the variables for each problem: NK-landscape uses 1-bit flip (binary encoding), TSP uses 2-opt (order-based permutation encoding), and QAP uses 2-swap (position-based permutation encoding) [23]. The knapsack problem requires a 2-bit flip neighbourhood, as 1-bit flip often fails to yield feasible or improved solutions near the constraint boundary, following the practice in this study [43].

We evaluate performance using the hypervolume (HV) indicator [72], with respect to a reference point estimated via random sampling. Specifically, we generate 100,000 random solutions from the decision space and only retain the non-dominated ones. According to [42], we define the reference point as $r_i = \max_i + (\max_i - \min_i)/10$ for minimisation problems, and $r_i = \min_i - (\max_i - \min_i)/10$ for maximisation problems, where $\max_i$ and $\min_i$ denote the maximum and minimum values of the non-dominated set on the i -th objective, respectively. This is more suitable than considering the common practice that uses a reference point determined by the combined non-dominated set of all generated solutions, because in multi-objective combinatorial optimisation different algorithms may exhibit substantially different levels of convergence (i.e., closeness to the Pareto front) [43].

Additional Results

This section gives additional experimental results on the four MOCOPs with the problem sizes of 200 and 500.

Figure 6 shows the average hypervolume (bolded line) and standard deviation (shaded area) of the basic s -PLS and r -PLS, as well as the two s -PLS variants across 30 independent runs on the four problems. The top row corresponds to problems with 200 variables, and the bottom row to those with 500 variables. Across all problem types, r -PLS consistently achieves higher hypervolume values than all s -PLS variants throughout the search. It also attains better final performance – the final HV values of r -PLS are significantly higher under the Wilcoxon rank-sum test at a 0.05 significance level. On the NK-landscape problem, the performance advantage of r -PLS is relatively smaller, while on the other three problems, r -PLS demonstrates a clear edge since the very beginning of the search process, particularly as the problem size increases. It is also worth noting that s -PLS $\prec$ and s -PLS $\nearrow$ (first-improvement variants) are faster than the original s -PLS (best-improvement) on most problems (except for the NK-landscape), which is in line with the previous findings [47, 21].

Figure 7 presents the non-dominated solutions obtained by the four algorithms in a typical run on the four problems when the search ends in the Figure 6. The layout mirrors that of Figure 6, with smaller instances shown in the top row and larger ones in the bottom row. For the Knapsack and NK-Landscape problems (maximisation), better solutions are located toward the top-right corner; for the TSP and QAP (minimisation), the desirable region lies in the bottom-left. As illustrated, r -PLS yields solutions with superior convergence and diversity compared to all the s -PLS variants. The original s -PLS, in particular, often exhibits significantly poorer spread across the objective space than the other algorithms.

It is worth pointing out that s -PLS exhibits poorer diversity on the TSP and QAP than on the NK-Landscape for all problem sizes (Figure 7 here and Figure 2 in the main paper). The systematic neighbourhood exploration of s -PLS evaluates all neighbours of a solution – its effectiveness depends on how diverse those neighbours are. On the NK-Landscape, random epistatic interactions [3] make the objective changes

of each bit-flip unpredictable, leading to distinct directions in the objective space even from a single solution. However, such diversity within a neighbourhood do not hold on the TSP and QAP.

The above results demonstrate that r -PLS achieves better performance in both convergence and diversity, with the advantage becoming more pronounced as the problem size increases.

Distribution of Solutions’ Neighbours in s -PLS and r -PLS

This section describes the details on investigating the distribution of the number of good neighbours for s -PLS and r -PLS. Specifically, we count the number of good neighbours among solutions in the archive during the search process of s -PLS and r -PLS on the four MOCOPs, and test if this number follows a known discrete probabilistic model. The distribution models [36] considered in this paper are the most commonly seen ones, characterised by their different probabilistic tail type: *uniform* (a balance distribution), *Zipf* (heavy-tailed with polynomial decay), *geometric* (light-tailed with exponential decay), *Poisson* (light-tailed with super-exponential decay), and *binomial* (light-tailed with bounded support and a hard cut-off) distributions.

Following the common practice, we evaluate the absolute fit quality using the χ^2 goodness-of-fit test at a significance level of $\alpha = 5\%$. A fit is considered acceptable if the test does not reject the null hypothesis that the observed data come from the tested distribution. Parameters of the distributions were estimated via maximum likelihood. For the Poisson distribution, the rate parameter λ was set to the sample mean. Similarly, for the geometric and binomial distributions, the success probability p was estimated from the sample mean as well. For the Zipf distribution, the exponent parameter s was estimated by numerically minimising the negative log-likelihood over the interval $[1.01, 10]$.

Figure 9 plots the goodness-of-fit of the five discrete distributions with respect to the numbers of good neighbours of solutions in the archive during the search process of s -PLS and r -PLS on the four MOCOPs. Each horizontal band corresponds to a candidate distribution, and at each sampled evaluation, two coloured ticks, black and blue, respectively indicate a good fit for s -PLS and r -PLS at $\alpha = 0.05$, namely, the χ^2 test does not reject the distribution. As can be seen in the figure, across nearly all problem instances, the uniform distribution only fits for s -PLS at the beginning of the search. In contrast, the geometric distribution passes the χ^2 test almost across all the band of the problems for both s -PLS and r -PLS, indicating that the number of good neighbours fits well with a geometric distribution.

Note that the Zipf distribution has a comparable number of fits to the geometric distribution across all problems in Figure 9. To compare the relative quality of fit between the two distributions, we use Akaike’s Information Criterion (AIC) [4], which balances goodness of fit with model complexity by penalising models with more free parameters. For each distribution, the degrees of freedom for each model are calculated as $n_{\text{bin}} - 1 - k$ where n_{bin} is the number of solutions that have at least one good neighbour and k is the

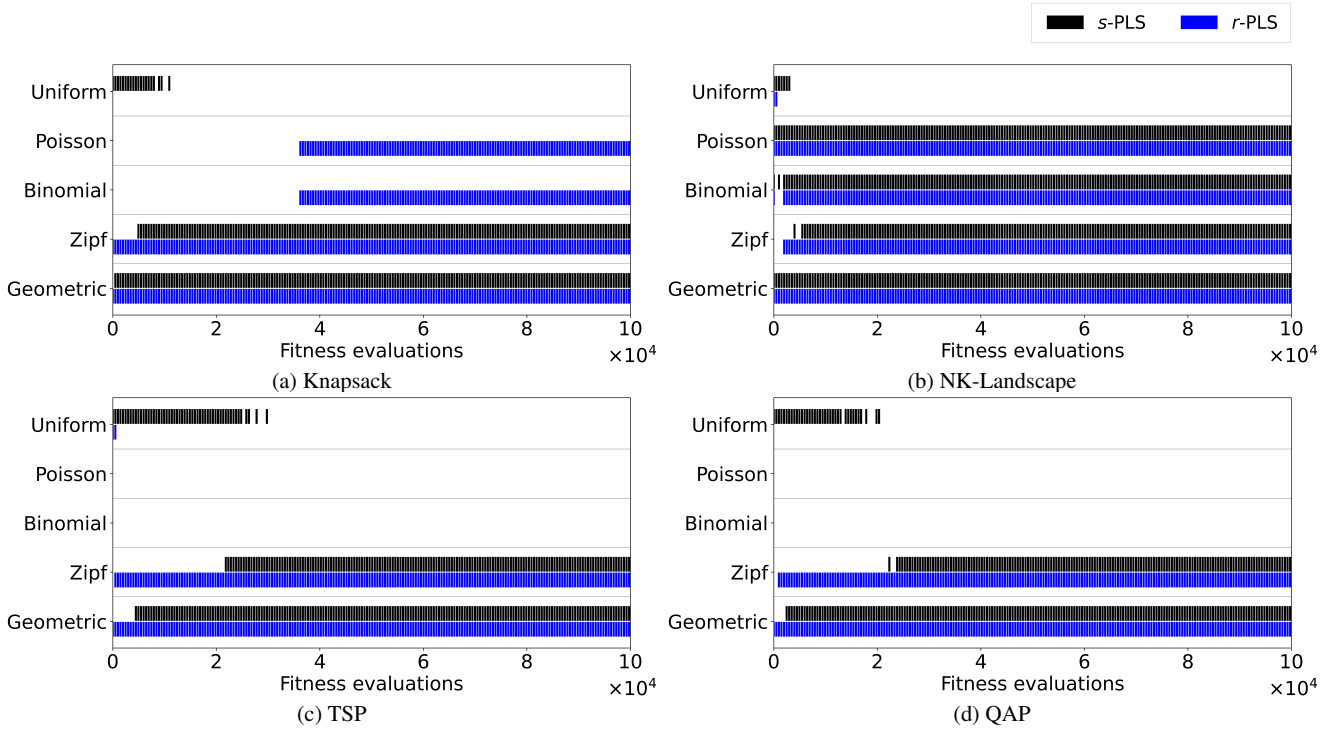


Figure 3: Goodness-of-fit of the distributions with respect to the number of good neighbours of solutions in the archive during the search process of *s*-PLS (black) and *r*-PLS (blue) on the (a) Knapsack (100D), (b) NK-Landscape (N=100, K=10), (c) TSP (100D) and (d) QAP (100D). A coloured tick in a row indicates that the corresponding algorithm's data at that point was not rejected under the model.

Figure 4: Goodness-of-fit of the distributions with respect to the number of good neighbours of solutions in the archive during the search process of *s*-PLS (black) and *r*-PLS (blue) on the (a) Knapsack (100D), (b) NK-Landscape (N=100, K=10), (c) TSP (100) and (d) QAP (100 factories). A coloured tick in a row indicates that the corresponding algorithm's data at that point was not rejected under the model.

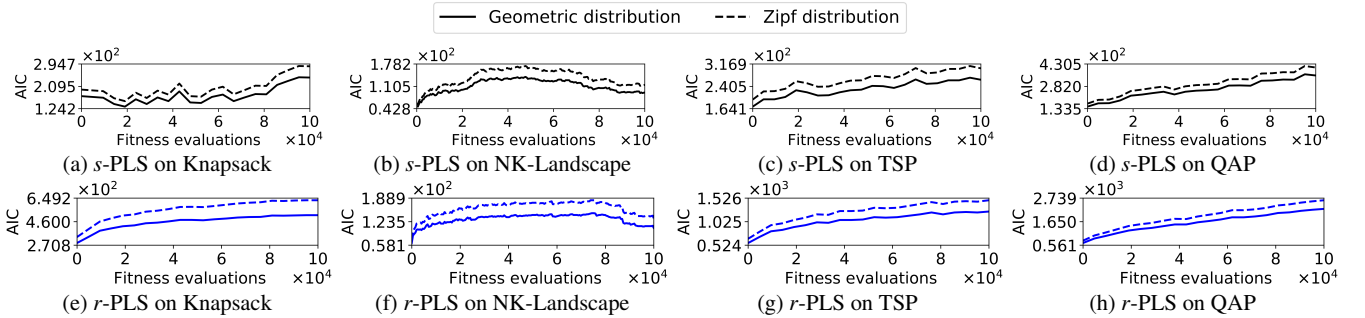


Figure 5: The Akaike's Information Criterion (AIC) [4] trajectories (lower is better) of the geometric distribution (solid line) and the Zipf distribution (dashed line) throughout the search processes of *s*-PLS (top panel) and *r*-PLS (bottom panel) on the four MOCOPs with 100 variables. In all sub-figures, the solid lines lie below the dashed lines, indicating that the geometric distribution consistently provides a better fit than the Zipf distribution.

number of free parameters of the distribution. A lower AIC indicates that the model fits better to the data. Figure 10 plots the AIC trajectory of the geometric and Zipf distributions throughout the search processes of *s*-PLS and *r*-PLS on the four problems. The solid and dashed lines correspond to the geometric and Zipf distributions, respectively. As shown, the solid line lies below the dashed line in all cases, indicating

that the geometric distribution consistently provides a better fit than the Zipf distribution throughout the search process in both *s*-PLS and *r*-PLS.

Proofs of the Two Examples

In the main paper, we present two toy examples of solution's distributions in the archive to help illustrate why *r*-PLS can

be faster than s -PLS. In Example 1, the archive contains two promising solutions (each having only good neighbours) and two unpromising solutions (each having no good neighbours). In Example 2, all the four solutions in the archive are half-promising (each having an equal mix of good and poor neighbours). Here, we are interested in how quickly (in terms of the number of evaluations) an algorithm can find a good neighbour. In this section, we provide the proofs showing that r -PLS is faster in Example 1, whereas s -PLS is faster in Example 2.

Proposition 1 (r -PLS is faster than s -PLS in Example 1). *Let an archive of n solutions contain exactly $n/2$ solutions with no good neighbours ($G = 0$), and the other $n/2$ solutions with all neighbours being good ($G = |\mathcal{N}|$). Then, for any neighbourhood size $|\mathcal{N}| \geq 2$, r -PLS requires less time (fewer number of evaluations) than s -PLS in finding the next new good solution.*

Proof. For r -PLS, the algorithm selects a solution uniformly at random from the archive. With probability $\frac{1}{2}$, it selects a solution whose entire neighbourhood consists of good solutions. Since neighbours are also sampled uniformly, any selected neighbour will be good. Therefore, the success probability is:

$$p_{\text{succ}} = \frac{1}{2}, \quad \text{and} \quad \mathbb{E}[T_{r\text{-PLS}}] = \frac{1}{p_{\text{succ}}} = \frac{1}{1/2} = 2. \quad (4)$$

In contrast, s -PLS scans the archive sequentially as it marks each visited solution as “explored”. For each unpromising solution (one without good neighbours), it evaluates all $|\mathcal{N}|$ neighbours before moving on. This continues until a promising solution is selected from the archive. This process is equivalent to the one described in Lemma 1 (i.e., finding the first good neighbour in the neighbourhood in a sequential scan) of the main paper. Accordingly, the expected number of evaluations required to find first promising solution is:

$$\mathbb{E}[J] = \frac{n+1}{(n/2)+1} = \frac{2(n+1)}{n+2}.$$

The total number of evaluations includes $|\mathcal{N}|$ evaluations for each of the $\mathbb{E}[J] - 1$ preceding unpromising solutions, and just 1 evaluation for the first promising one (since all of its neighbours are good). Therefore:

$$\begin{aligned} \mathbb{E}[T_{s\text{-PLS}}] &= (\mathbb{E}[J] - 1)|\mathcal{N}| + 1 \\ &= \left(\frac{2(n+1)}{n+2} - 1 \right) |\mathcal{N}| + 1 = \frac{n|\mathcal{N}|}{n+2} + 1. \end{aligned} \quad (5)$$

Since $|\mathcal{N}| \geq 2$, it follows that $\mathbb{E}[T_{s\text{-PLS}}] > \mathbb{E}[T_{r\text{-PLS}}]$. $\square$

The above proposition shows that the r -PLS is faster than s -PLS in finding the first good solution in Example 1. The key reason is that s -PLS may waste a substantial number of evaluations when it selects an unpromising solution, as it evaluates all of its neighbours. In contrast, r -PLS avoids this by sampling only one neighbour at a time. In Example 2, however, the archive contains no unpromising solutions; instead, all solutions are half-promising – each has an equal mix of good and poor neighbours. In what follows, we show that in Example 2, s -PLS is faster than r -PLS.

Proposition 2 (s -PLS is faster than r -PLS in Example 2). *Let the archive consist of n solutions, and suppose that for every solution, exactly half of its neighbours are good; that is, $G = |\mathcal{N}|/2$. Then, for any neighbourhood size $|\mathcal{N}| \geq 2$, s -PLS requires less time (fewer number of evaluations) than r -PLS in finding the next new good solution.*

Proof. In r -PLS, a solution is selected uniformly at random from the archive, and a neighbour is drawn uniformly from its neighbourhood. Since each solution has exactly half good neighbours, the probability of success (i.e., selecting a good neighbour) is:

$$p_{\text{succ}} = \frac{1}{2}, \quad \text{and} \quad \mathbb{E}[T_{r\text{-PLS}}] = \frac{1}{p_{\text{succ}}} = \frac{1}{1/2} = 2. \quad (6)$$

As for s -PLS, the algorithm explores the neighbourhood of one solution at a time, scanning its neighbours sequentially until a good one is found. Since all solutions in the archive have identical neighbourhood structure, it suffices to consider the expected number of evaluations spent within a single neighbourhood. In this case, s -PLS only needs to scan one neighbourhood as all neighbourhoods have $|\mathcal{N}|/2$ good neighbours. By Lemma 1 in the main paper, when half of the neighbours are good, the expected number of evaluations required to find a good neighbour within one neighbourhood is:

$$\mathbb{E}[T_{s\text{-PLS}}] = \mathbb{E}[J] = \frac{|\mathcal{N}| + 1}{(|\mathcal{N}|/2) + 1} < 2 = \mathbb{E}[T_{r\text{-PLS}}].$$

$\square$

Proof for r -PLS with Best-from-Multiple Selections

In the main paper, we state that the use of Best-from-Multiple-Selections (BMS) [11] in r -PLS requires $\mathbb{E}[T_{r\text{-PLS}}] \cdot (1 + O(k/|\mathcal{N}|))$ evaluations to find the next good neighbour. In this section, we provide the proof of that.

Consider a variant of r -PLS equipped with Best-from-Multiple-Selections (BMS): in each iteration, a solution is drawn uniformly at random from the archive, and up to k neighbours ($1 \leq k \leq |\mathcal{N}|$) of this solution are sampled uniformly without replacement and evaluated one-by-one, stopping as soon as a good neighbour is found or after k evaluations if no good neighbour is encountered. Let T_{BMS} denote the number of evaluations required to find the next good neighbour.

Proposition 3 (Expected time of r -PLS with Best-from-Multiple-Selections). *Let $|\mathcal{N}| \in \mathbb{Z}^+$ be the neighbourhood size and let G be the number of good neighbours among the solutions in the archive, where G follows a geometric distribution with parameter $p \in (0, 1]$. Let T_{BMS} be the number of evaluations until the next good neighbour is found by the BMS variant of r -PLS. Then*

$$\mathbb{E}[T_{\text{BMS}}] > \mathbb{E}[T_{r\text{-PLS}}]$$

where $\mathbb{E}[T_{r\text{-PLS}}] = p|\mathcal{N}|/(1-p)$.

Proof. Recalling r -PLS from the main theorem, conditional on G , the success probability is given by

$$q := \mathbb{E}\left[\frac{G}{|\mathcal{N}|}\right] = \frac{\mathbb{E}[G]}{|\mathcal{N}|} = \frac{1-p}{p|\mathcal{N}|},$$

and $\mathbb{E}[T_{r\text{-PLS}}] = 1/q = p|\mathcal{N}|/(1-p)$.

We next analyse the BMS variant, which samples $k > 1$ neighbours of a solution at a time. Regarding the selected solution, there are typically two cases: $G = 0$ (no good neighbour) and $G > 0$ (at least one good neighbour).

We start from considering the $G > 0$ case. At the beginning of each BMS iteration, a solution with $G = g$ good neighbours is chosen. Within that iteration, suppose h neighbours have already been examined and found poor ($0 \leq h \leq k-1$). The next evaluation chooses uniformly one neighbour among the remaining $|\mathcal{N}| - h$ neighbours, which still contain the same g good ones (because the iteration would have stopped earlier otherwise). Thus, conditional on $G = g$,

$$\Pr(\text{success at this evaluation} \mid G = g, h) = \frac{g}{|\mathcal{N}| - h}.$$

Since $0 \leq h \leq k-1 \leq |\mathcal{N}| - 1$, we have

$$|\mathcal{N}| - (k-1) \leq |\mathcal{N}| - h \leq |\mathcal{N}|,$$

and therefore

$$\frac{g}{|\mathcal{N}|} \leq \frac{g}{|\mathcal{N}| - h} \leq \frac{g}{|\mathcal{N}| - (k-1)}.$$

With $0 \leq h \leq k-1$, we have the bounds on the success probability q_t of the t -th evaluation in the BMS process (given that we have not succeeded earlier):

$$\mathbb{E}[G]/|\mathcal{N}| \leq q_t \leq \frac{\mathbb{E}[G]}{|\mathcal{N}| - (k-1)} \quad \forall t \geq 1.$$

Plugging in the success probability of r -PLS, we have

$$q \leq q_t \leq \frac{q}{1 - \frac{k-1}{|\mathcal{N}|}} \quad \forall t \geq 1.$$

We now bound the stopping time T_{BMS} from the bounds on q_t . Let $q_{\min} = q$ and $q_{\max} = q/(1 - \frac{k-1}{|\mathcal{N}|})$. For the BMS process, write q_t for the success probability of the t -th evaluation (conditioned on no success earlier), we have

$$\frac{1}{q_{\max}} \leq \frac{1}{q_t} \leq \frac{1}{q_{\min}} \quad \forall t \geq 1.$$

Substituting the equations with the expected number of evaluations until a good neighbour is found $\mathbb{E}[T_{\text{bms}}]$,

$$\frac{1}{q_{\max}} \leq \mathbb{E}[T_{\text{bms}}] \leq \frac{1}{q_{\min}}.$$

Plugging $q_{\min}$ and $q_{\max}$ back, we obtain

$$\left(1 - \frac{k-1}{|\mathcal{N}|}\right) \mathbb{E}[T_{r\text{-PLS}}] \leq \mathbb{E}[T_{\text{bms}}] \leq \mathbb{E}[T_{r\text{-PLS}}].$$

Dividing by $\mathbb{E}[T_{r\text{-PLS}}]$ yields

$$1 - \frac{k-1}{|\mathcal{N}|} \leq \frac{\mathbb{E}[T_{\text{bms}}]}{\mathbb{E}[T_{r\text{-PLS}}]} \leq 1,$$

Then, we add the case of no good neighbours ($G = 0$) that wastes k evaluations at the rate p out of the $\frac{1}{1-p}$ solutions explored, which yields

$$\mathbb{E}[T_{\text{BMS}}] = \frac{kp}{1-p} + \mathbb{E}[T_{\text{bms}}].$$

Plugging in the above inequalities, we have

$$\begin{aligned} \frac{kp}{1-p} + \left(1 - \frac{k-1}{|\mathcal{N}|}\right) \mathbb{E}[T_{r\text{-PLS}}] &\leq \mathbb{E}[T_{\text{BMS}}] \leq \frac{kp}{1-p} + \mathbb{E}[T_{r\text{-PLS}}] \\ \mathbb{E}[T_{r\text{-PLS}}] + \frac{\mathbb{E}[T_{r\text{-PLS}}]}{|\mathcal{N}|} &\leq \mathbb{E}[T_{\text{BMS}}] \leq \frac{k}{|\mathcal{N}|} \mathbb{E}[T_{r\text{-PLS}}] + \mathbb{E}[T_{r\text{-PLS}}] \\ 1 + \frac{1}{|\mathcal{N}|} &\leq \frac{\mathbb{E}[T_{\text{BMS}}]}{\mathbb{E}[T_{r\text{-PLS}}]} \leq 1 + \frac{k}{|\mathcal{N}|} \end{aligned}$$

Since $1 + \frac{1}{|\mathcal{N}|} \leq \frac{\mathbb{E}[T_{\text{BMS}}]}{\mathbb{E}[T_{r\text{-PLS}}]}$, the BMS variant is slower than the r -PLS. $\square$

It is worth noting that, since $\frac{\mathbb{E}[T_{\text{BMS}}]}{\mathbb{E}[T_{r\text{-PLS}}]} \leq 1 + \frac{k}{|\mathcal{N}|}$, the BMS variant is only slightly slower than the r -PLS when k is small.

References

- [1] Hernán Aguirre and Kiyoshi Tanaka. Working principles, behavior, and performance of MOEAs on MNK-landscapes. *Eur. J. Oper. Res.*, 181(3):1670–1690, 2007.
- [2] Hernan Aguirre and Kiyoshi Tanaka. Random bit climbers on multiobjective mnk-landscapes: effects of memory and population climbing. *IEICE Trans. Fundam. Electron. Commun. Comput. Sci.*, 88(1):334–345, 2005.
- [3] Hernán E Aguirre and Kiyoshi Tanaka. Effects of elitism and population climbing on multiobjective mnk-landscapes. In *CEC*, pages 449–456, 2004.
- [4] Hirotugu Akaike. A new look at the statistical model identification. *IEEE Trans. Autom. Control*, 19(6):716–723, 1974.
- [5] Abdullah Alsheddy and Edward E. P. K Tsang. Guided pareto local search based frameworks for biobjective optimization. In *CEC*, pages 1–8, 2010.
- [6] Nawal Benabbou, Cassandre Leroy, Thibaut Lust, and Patrice Perny. Combining preference elicitation with local search and greedy search for matroid optimization. In *AAAI*, pages 12233–12240, 2021.
- [7] Nicola Beume, Boris Naujoks, and Michael Emmerich. SMS-EMOA: Multiobjective selection based on dominated hypervolume. *Eur. J. Oper. Res.*, (3):1653–1669, 2007.
- [8] Chao Bian, Chao Qian, and Ke Tang. A general approach to running time analysis of multi-objective evolutionary algorithms. In *IJCAI*, pages 1405–1411, 2018.
- [9] Aymeric Blot, Marie-Éléonore Kessaci, and Laetitia Jourdan. Survey and unification of local search techniques in metaheuristics for multi-objective combinatorial optimisation. *Journal of Heuristics*, 24(6):853–877, 2018.
- [10] L. Breslau, Pei Cao, Li Fan, G. Phillips, and S. Shenker. Web caching and Zipf-like distributions: evidence and implications. In *Annual Joint Conference*

of the *IEEE Computer and Communications Societies*, pages 126–134, 1999.

- [11] Shaowei Cai. Balance between complexity and quality: local search for minimum vertex cover in massive graphs. In *AAAI*, page 747–753, 2015.
- [12] Sara Ceschia, Luca Di Gaspero, and Andrea Schaerf. Educational timetabling: Problems, benchmarks, and state-of-the-art results. *Eur. J. Oper. Res.*, 308(1):1–18, 2023.
- [13] Jiejiang Chen, Shaowei Cai, Shiwei Pan, Yiyuan Wang, Qingwei Lin, Mengyu Zhao, and Minghao Yin. Nuqclq: An effective local search algorithm for maximum quasi-clique problem. In *AAAI*, pages 12258–12266, 2021.
- [14] Francisco Chicano, Darrell Whitley, and Renato Tinos. Efficient hill climber for constrained pseudo-Boolean optimization problems. In *GECCO*, pages 309–316, 2016.
- [15] Fabio Daolio, Arnaud Liefooghe, Sébastien Verel, Hernán Aguirre, and Kiyoshi Tanaka. Global vs local search on multi-objective nk-landscapes: contrasting the impact of problem features. In *GECCO*, pages 369–376, 2015.
- [16] Kalyanmoy Deb, Amrit Pratap, Sameer Agarwal, and T Meyarivan. A fast and elitist multiobjective genetic algorithm: Nsga-ii. *IEEE Trans. Evol. Comp.*, 6(2):182–197, 2002.
- [17] Bilel Derbel, Arnaud Liefooghe, Qingfu Zhang, Hernan Aguirre, and Kiyoshi Tanaka. Multi-objective local search based on decomposition. In *PPSN*, pages 431–441, 2016.
- [18] M. Dinot, B. Doerr, U. Hennebelle, and Will S. Runtime analyses of multi-objective evolutionary algorithms in the presence of noise. In *IJCAI*, pages 5549–5557, 2023.
- [19] Benjamin Doerr and Weijie Zheng. Theoretical analyses of multi-objective evolutionary algorithms on multi-modal objectives. In *AAAI*, pages 12293–12301, 2021.
- [20] Mădălina M Drugan and Dirk Thierens. Stochastic pareto local search: Pareto neighbourhood exploration and perturbation strategies. *Journal of Heuristics*, 18:727–766, 2012.
- [21] Jérémie Dubois-Lacoste, Manuel López-Ibáñez, and Thomas Stützle. Anytime pareto local search. *Eur. J. Oper. Res.*, 243(2):369–385, 2015.
- [22] Juan J. Durillo and Antonio J. Nebro. jMetal: A Java framework for multi-objective optimization. *Advances in Engineering Software*, 42(10):760–771, 2011.
- [23] Agoston E Eiben and James E Smith. *Introduction to Evolutionary Computing*. Springer, 2015.
- [24] José Rui Figueira, Carlos M Fonseca, Pascal Halffmann, Kathrin Klamroth, Luís Paquete, Stefan Ruzika, Britta Schulze, Michael Stiglmayr, and David Willems. Easy to say they are hard, but hard to see they are easy—towards a categorization of tractable multiobjective combinatorial optimization problems. *Journal of Multi-Criteria Decision Analysis*, 24(1-2):82–98, 2017.
- [25] Mengxing Gao, ChenGuang Liu, and Xi Chen. A bi-objective unrelated parallel machine scheduling problem with additional resources and soft precedence constraints. *Eur. J. Oper. Res.*, 325(1):53–66, 2025.
- [26] Jaroslav Garvardt, Niels Grüttemeier, Christian Komusiewicz, and Nils Morawietz. Parameterized local search for max c -cut. In *IJCAI*, page 5586–5594, 2024.
- [27] O. Giel. Expected runtimes of a simple multi-objective evolutionary algorithm. In *CEC*, pages 1918–1925, 2003.
- [28] Oliver Giel and Per Kristian Lehre. On the effect of populations in evolutionary multi-objective optimization. In *GECCO*, pages 651–658, 2006.
- [29] Niels Grüttemeier, Christian Komusiewicz, and Nils Morawietz. Efficient bayesian network structure learning via parameterized local search on topological orderings. In *AAAI*, pages 12328–12335, 2021.
- [30] Zhenyu Guo, Mingyu Xiao, Yi Zhou, Dongxiang Zhang, and Kian-Lee Tan. Enhancing balanced graph edge partition with effective local search. In *AAAI*, pages 12336–12343, 2021.
- [31] Felipe Honjo Ide, Hernan Aguirre, and Kiyoshi Tanaka. Multi-objective random bit climbers with weighted permutation on large scale binary MNK-landscapes. In *PPSN*, pages 169–185, 2024.
- [32] Maarten Inja, Chiel Kooijman, Maarten de Waard, Diederik M Roijers, and Shimon Whiteson. Queued Pareto local search for multi-objective optimization. In *PPSN*, pages 589–599, 2014.
- [33] Hisao Ishibuchi and Tadahiko Murata. A multi-objective genetic local search algorithm and its application to flowshop scheduling. *IEEE Trans. Syst., Man, Cybern.*, 28(3):392–403, 1998.
- [34] Andrzej Jaskiewicz. On the performance of multiple-objective genetic local search on the 0/1 knapsack problem - a comparative experiment. *IEEE Trans. Evol. Comp.*, 6(4):402–412, 2002.
- [35] Andrzej Jaskiewicz. Many-objective pareto local search. *Eur. J. Oper. Res.*, 271(3):1001–1013, 2018.
- [36] Norman L. Johnson, Samuel Kotz, and Adrienne W. Kemp. *Univariate Discrete Distributions*. John Wiley & Sons, 3rd edition, 2005.
- [37] Yuhao Kang, Jialong Shi, Jianyong Sun, Qingfu Zhang, and Ye Fan. New techniques to improve neighborhood exploration in Pareto local search. *Expert Syst. Appl.*, 254:124296, 2024.
- [38] Joshua Knowles and David Corne. Instance generators and test suites for the multiobjective quadratic assignment problem. In *EMO*, pages 295–310, 2003.
- [39] Donald E. Knuth. *The Art of Computer Programming, Volume 1: Fundamental Algorithms*. Addison-Wesley, 1997.

- [40] M. Laumanns, L. Thiele, and E. Zitzler. Running time analysis of multiobjective evolutionary algorithms on pseudo-Boolean functions. *IEEE Trans. Comput.*, 8(2):170–182, 2004.
- [41] Marco Laumanns, Lothar Thiele, Kalyanmoy Deb, and Eckart Zitzler. Combining convergence and diversity in evolutionary multiobjective optimization. *Evol. Comput.*, 10(3):263–282, 2002.
- [42] Miqing Li, Tao Chen, and Xin Yao. How to evaluate solutions in Pareto-based search-based software engineering? A critical review and methodological guidance. *IEEE Trans. Evol. Comput. on Software Engineering*, 48(5):1771–1799, 2020.
- [43] Miqing Li, Xiaofeng Han, Xiaochen Chu, and Zimin Liang. Empirical comparison between MOEAs and local search on multi-objective combinatorial optimisation problems. In *GECCO*, pages 547–556, 2024.
- [44] Miqing Li and Xin Yao. Quality evaluation of solution sets in multiobjective optimisation: A survey. *ACM Computing Surveys*, (2), 2019.
- [45] Jiaxin Liang, Junping Zhou, and Minghao Yin. Diversat: A novel and effective local search algorithm for diverse SAT problem. In *AAAI*, pages 11290–11298, 2025.
- [46] Zimin Liang and Miqing Li. On the problem characteristics of multi-objective pseudo-Boolean functions in runtime analysis. In *Foundations of Genetic Algorithms*, page 166–177, 2025.
- [47] Arnaud Liefooghe, Jérémie Humeau, Salma Mesmoudi, Laetitia Jourdan, and El-Ghazali Talbi. On dominance-based multiobjective local search: design, implementation and experimental analysis on scheduling and traveling salesman problems. *Journal of Heuristics*, 18:317–352, 2012.
- [48] Peng Lin, Mengchuan Zou, Zhihan Chen, and Shaowei Cai. Parailp: A parallel local search framework for integer linear programming with cooperative evolution mechanism. In *IJCAI*, pages 6949–6957, 2024.
- [49] Thibaut Lust and Jacques Teghem. Two-phase pareto local search for the biobjective traveling salesman problem. *Journal of Heuristics*, 16(3):475–510, 2010.
- [50] Thibaut Lust and Jacques Teghem. The multiobjective multidimensional knapsack problem: a survey and a new approach. *Int. Trans. Oper. Res.*, 19(4):495–520, 2012.
- [51] Luis Paquete, Marco Chiarandini, and Thomas Stützle. Pareto local optimum sets in the biobjective traveling salesman problem: An experimental study. In *Metaheuristics for Multiobjective Optimisation*, pages 177–199, 2004.
- [52] Luis Paquete and Thomas Stützle. A two-phase local search for the biobjective traveling salesman problem. In *EMO*, pages 479–493, 2003.
- [53] Quan Minh Phan and Ngoc Hoang Luong. Pareto local search is competitive with evolutionary algorithms for multi-objective neural architecture search. In *GECCO*, pages 348–356, 2023.
- [54] Zhongqiang Ren, Akshaya Kesarimangalam Srinivasan, Bhaskar Vundurthy, Ian Abraham, and Howie Choset. A Pareto-optimal local optimization framework for multiobjective ergodic search. *IEEE Trans. Robot.*, 39(5):3452–3463, 2023.
- [55] Celso C Ribeiro, Pierre Hansen, Pedro Castro Borges, and Michael Pilegaard Hansen. A study of global convexity for a multiple objective travelling salesman problem. *Essays and Surveys in Metaheuristics*, pages 129–150, 2002.
- [56] Vívian Ludimila Aguiar Santos, Thales Francisco Mota Carvalho, Luciana Pereira de Assis, Miri Weiss-Cohen, and Frederico Gadelha Guimarães. Multi-objective iterated local search based on decomposition for job scheduling problems with machine deterioration effect. *Engineering Applications of Artificial Intelligence*, 112:104826, 2022.
- [57] Swakkhar Shatabda, MA Newton, and Abdul Sattar. Mixed heuristic local search for protein structure prediction. In *AAAI*, pages 876–882, 2013.
- [58] Jialong Shi, Jianyong Sun, Qingfu Zhang, Haotian Zhang, and Ye Fan. Improving pareto local search using cooperative parallelism strategies for multiobjective combinatorial optimization. *IEEE Trans. Cybern.*, pages 2369–2382, 2022.
- [59] Jialong Shi, Qingfu Zhang, and Jianyong Sun. PPLS/D: Parallel Pareto local search based on decomposition. *IEEE Trans. Cybern.*, 50(3):1060–1071, 2020.
- [60] Zhouxing Su, Qingyun Zhang, Zhipeng Lü, Chu-Min Li, Weibo Lin, and Fuda Ma. Weighting-based variable neighborhood search for optimal camera placement. In *AAAI*, pages 12400–12408, 2021.
- [61] Rui Sun, Yiyuan Wang, HL Shimao Wang, Hui Li, Ximing Li, and Minghao Yin. Nukplex: An efficient local search algorithm for maximum k-plex problem. In *IJCAI*, pages 7029–7037, 2024.
- [62] Jacques Teghem. Multi-objective combinatorial optimization problems: A survey. *Journal of Multi-Criteria Decision Analysis*, 3:83–104, 1994.
- [63] Sébastien Verel, Arnaud Liefooghe, Laetitia Jourdan, and Clarisse Dhaenens. On the structure of multiobjective combinatorial search space: Mnk-landscapes with correlated objectives. *Eur. J. Oper. Res.*, 227(2):331–342, 2013.
- [64] S. Wietheger and B. Doerr. Near-tight runtime guarantees for many-objective evolutionary algorithms. In *PPSN*, pages 153–168, 2024.
- [65] Zheng Xie, Zhenzheng Ouyang, Qi Liu, and Jianping Li. A geometric graph model for citation networks of exponentially growing scientific papers. *Physica A: Statistical Mechanics and its Applications*, 456:167–175, 2016.
- [66] Zhenxuan Xie, Zhipeng Lü, Zhouxing Su, Chu-Min Li, Junwen Ding, and Yuxuan Wang. A swap relaxation-based local search for the latin square completion problem. In *IJCAI*, pages 7047–7055, 2024.

- [67] Furong Ye, Chuan Luo, and Shaowei Cai. Better understandings and configurations in maxsat stochastic local search solvers via anytime performance analysis. In *AAAI*, pages 27153–27160, 2025.
- [68] Qingfu Zhang and Hui Li. MOEA/D: A multiobjective evolutionary algorithm based on decomposition. *IEEE Trans. Evol. Comp.*, 11(6):712–731, 2007.
- [69] Zhen Zhang, Junfeng Yang, Limei Liu, Xuesong Xu, Guozhen Rong, and Qilong Feng. Towards a theoretical understanding of why local search works for clustering with fair-center representation. In *AAAI*, pages 16953–16960, 2024.
- [70] Jiongzhi Zheng, Kun He, Jianrong Zhou, Yan Jin, Chu-Min Li, and Felip Manyà. Integrating multi-armed bandit with local search for maxsat. *Artificial Intelligence*, 338:104242, 2025.
- [71] W. Zheng, M. Li, R. Deng, and B. Doerr. How to use the metropolis algorithm for multi-objective optimization? In *AAAI*, pages 20883–20891, 2024.
- [72] Eckart Zitzler and Lothar Thiele. Multiobjective evolutionary algorithms: A comparative case study and the strength Pareto approach. *IEEE Trans. Evol. Comp.*, 3(4):257–271, 1999.